

UNIT 1 – Software Testing Fundamentals

Exhaustive Study Notes covering Testing Principles, SDLC, STLC, Test Cases & Defect Life Cycle

1. Testing Basics

Software Testing is the process of executing a program or system with the intent of finding errors, verifying that it meets specified requirements, and validating that it satisfies user expectations.

Core Distinction: • Quality Assurance (QA): Process-oriented. Focuses on preventing defects in the software development process. • Quality Control (QC): Product-oriented. Focuses on identifying defects in the final software product. • Software Testing: Activity of executing software to evaluate quality and detect bugs.

7 Principles of Software Testing: 1. Testing shows the presence of defects, not their absence. 2. Exhaustive testing is impossible. 3. Early testing saves time and money. 4. Defect clustering (80% of defects found in 20% of modules). 5. Pesticide Paradox (Repeat tests lose effectiveness over time). 6. Testing is context-dependent (E-commerce vs Healthcare app). 7. Absence-of-errors fallacy (Fixing bugs doesn't help if system is unusable).

■ **Key Takeaway:** Early testing reduces bug fixing cost exponentially compared to finding bugs in production.

2. SDLC / STLC

• SDLC (Software Development Life Cycle): Framework defining steps in software development (Requirement Analysis -> Design -> Coding -> Testing -> Deployment -> Maintenance). • STLC (Software Testing Life Cycle): Specific sequence of testing activities executed to meet quality goals: 1. Requirement Analysis: Identify testable requirements. 2. Test Planning: Strategy, scope, resource estimation, tool selection. 3. Test Case Development: Writing detailed test cases and test data. 4. Test Environment Setup: Hardware/Software environment deployment. 5. Test Execution: Execute test cases, compare actual vs expected results, log bugs. 6. Test Cycle Closure: Metrics evaluation, test closure report, sign-off.

■ **Key Takeaway:** STLC runs parallel to SDLC. Test planning begins as soon as requirements are finalized.

Software Testing Life Cycle (STLC) Phases



Diagram: Software Testing Life Cycle (STLC) Execution Phases Flowchart.

3. SDLC Models & V-Model

SDLC Models dictate development workflow: • Waterfall Model: Sequential phase execution. Low flexibility for changing requirements. • Agile / Scrum: Iterative and incremental development in short 2-4 week Sprints. • V-Model (Verification & Validation): Demonstrates relationship between each phase of SDLC and its corresponding testing phase in STLC.

■ **Key Takeaway:** In V-Model, Verification checks 'Are we building the product right?' while Validation checks 'Are we building the right product?'.

V-Model (Verification & Validation Architecture)

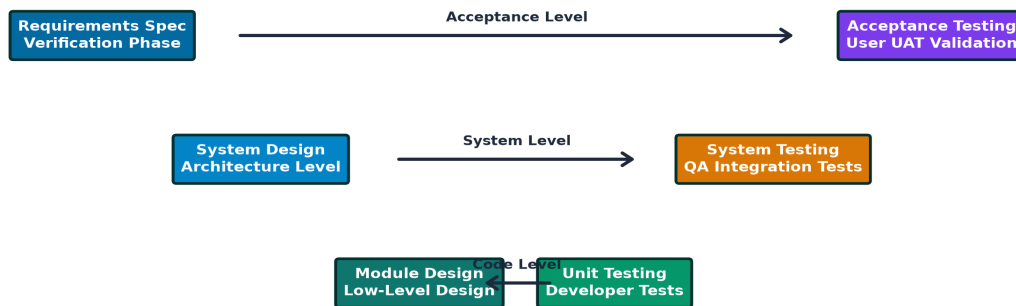


Diagram: V-Model Architecture showing Verification vs Validation alignment.

4. Test Cases & Test Scenarios

• Test Scenario: High-level classification of what to test (e.g., 'Verify User Login Functionality'). • Test Case: Detailed set of conditions, inputs, execution steps, preconditions, and expected results designed to verify a specific feature.

Anatomy of a Standard Test Case: 1. Test Case ID (e.g. TC_LOGIN_001) 2. Test Summary / Description 3. Preconditions 4. Test Data 5. Execution Steps 6. Expected Result 7. Actual Result (Pass / Fail) 8. Severity & Priority

```
# Sample Test Case Structure
Test Case ID: TC_LOGIN_001
Title: Verify login with valid credentials
Precondition: User is registered and on /login page
Test Steps:
  1. Enter valid email 'user@example.com'
  2. Enter valid password 'Password123!'
  3. Click 'Login' button
Expected Result: Redirect to /dashboard and display 'Welcome User'
Actual Result: User redirected to /dashboard successfully (PASS)
```

5. Bug / Defect Life Cycle

A Bug or Defect is a flaw in software that causes it to produce an incorrect or unexpected result.

Defect Life Cycle States: • NEW: Defect logged by tester. • ASSIGNED: Assigned to developer. • OPEN: Developer analyzing/fixing defect. • FIXED: Developer submits code fix. • RETEST: Tester re-runs test case on new build. • VERIFIED / CLOSED: Fix confirmed; defect closed. • REOPENED: Fix failed; sent back to developer. • REJECTED / DEFERRED: Invalid bug or delayed to future release.

Severity vs Priority: • Severity: Impact on software functionality (Critical, High, Medium, Low). • Priority: Urgency to fix bug (P1-Immediate, P2-High, P3-Normal).

■ **Key Takeaway:** Example of High Severity / Low Priority: Application crashes when clicking a rarely used link in Terms & Conditions.

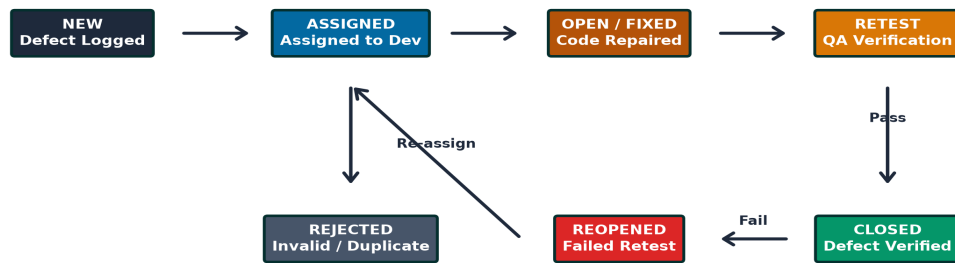
Defect / Bug Life Cycle State Machine Diagram

Diagram: Defect / Bug Life Cycle State Machine Diagram.